Project Guide — SBMS (Frontend + Backend)

This document explains the project structure, how CRUD works end-to-end, database connectivity, how to run the app, and includes a short VIVA-ready summary you can paste into your report.

Contents

- Project overview
- Files to cite
- Frontend: API & UI
- Backend: Controllers, Services, Repositories, Entities
- Database connectivity & setup
- How a CRUD request flows (step-by-step)
- Run & dev commands
- VIVA Q&A (paste-ready answers)
- Troubleshooting & notes

Project overview

This is a two-tier application:

- Frontend: Next.js React app (UI + API helpers)
- Backend: Spring Boot REST API (controllers → services → repositories)
- Database: MySQL with JPA/Hibernate ORM mapping

Frontend code calls a single API bridge module which contains the CRUD helpers; the backend exposes REST endpoints implemented as Spring controllers which use a service layer and JPA repositories to persist `@Entity` objects into MySQL.

Key files to cite in your report

- Frontend API bridge: frontend/lib/api.ts
- Example frontend UI page (user management): frontend/app/admin/users/page.tsx
- Backend controllers (API endpoints):
  - backend/src/main/java/com/restroaudit/backend/controller/UserController.java
  - backend/src/main/java/com/restroaudit/backend/controller/OrderController.java
  - backend/src/main/java/com/restroaudit/backend/controller/MenuItemController.java
- Backend service example: backend/src/main/java/com/restroaudit/backend/service/UserService.java
- Repository example: backend/src/main/java/com/restroaudit/backend/repository/UserRepository.java
- Entity folder: backend/src/main/java/com/restroaudit/backend/model
- DB config: backend/src/main/resources/application.properties

Frontend — API & UI

Main API helper: `frontend/lib/api.ts` centralizes `fetch()` calls and exposes functions such as:

- `fetchUsers()`, `registerUser()` — user CRUD

- `fetchOrders()`, `createOrder()`, `updateOrder()` — order flow
- `fetchMenuItems()`, `deleteMenuItem()` — menu item CRUD

This file ensures all requests use JSON and forwards a branch header when available:

```
// example call
await fetch(`${API_URL}/users`, { method: 'POST', body: JSON.stringify(payload) })
```

UI pages import these helpers. Example: the admin users page (`frontend/app/admin/users/page.tsx`) calls `fetchUsers()` on mount and `registerUser()` when creating.

Why include only `lib/api.ts` in the report? It neatly demonstrates how the frontend calls the backend for all CRUD operations.

Backend — Controllers, Services, Repositories, Entities

Pattern used throughout the backend:

1. Controller — annotated with `@RestController` and `@RequestMapping("/api/...")`. Defines endpoints using `@GetMapping`, `@PostMapping`, `@PutMapping`, `@DeleteMapping`.
    - Example: `UserController` maps `/api/users` and implements the full CRUD surface.
2. Service — annotated with `@Service`. Encapsulates business logic and transactional operations. Example: `OrderService` performs validation, saves the order, updates loyalty points and updates ingredient stock.
3. Repository — extends `JpaRepository<Entity, IdType>`. Provides `save()`, `findById()`, `findAll()`, `deleteById()`.
4. Entity — annotated with `@Entity` and `@Table(name = "...")`. Fields use `@Column`, relationships use `@ManyToOne` / `@OneToMany` and `@JoinColumn`.

Files: see the key file list above. The `model` folder currently contains 10 `@Entity` classes (User, Order, OrderDetail, MenuItem, Ingredient, Feedback, Department, Branch, AuditLog, ComplaintLog).

Note about missing tables: if your DB has more tables (e.g., `recipe`), some are used as lookup or mapping tables and may be accessed via `JdbcTemplate` (see `OrderService`) instead of an `@Entity`.

Database connectivity & setup

The connection is configured in `backend/src/main/resources/application.properties`:

```
spring.datasource.url=jdbc:mysql://localhost:3306/food_quality_db
spring.datasource.username=root
spring.datasource.password=<your-db-password>
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

Quick SQL to create the database (run in MySQL):

```
CREATE DATABASE IF NOT EXISTS food_quality_db CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;
-- create a user (optional)
CREATE USER 'sbms_user'@'localhost' IDENTIFIED BY 'secure_password';
GRANT ALL ON food_quality_db.* TO 'sbms_user'@'localhost';
FLUSH PRIVILEGES;
```

Replace credentials in `application.properties`. For production, avoid plaintext credentials — use environment variables or a secrets manager.

How a CRUD request flows (step-by-step)

Example: Create user

1. Frontend: `registerUser(payload)` in `frontend/lib/api.ts` sends POST `/api/users` with JSON body.
2. Backend controller: `UserController.create(@RequestBody User entity)` receives the request; Spring MVC deserializes JSON into `User` object.
3. Controller calls `userService.save(entity)`.
4. Service: `UserService.save()` may validate and then call `userRepository.save(entity)`.
5. Repository (JPA/Hibernate) converts entity → SQL `INSERT` and persists into `users` table.
6. Controller returns a DTO or map (flattened fields) to the frontend.

Run & dev commands

Backend (Maven):

```
cd backend
mvn spring-boot:run
```

Frontend (pnpm / npm):

```
cd frontend
pnpm install
pnpm dev
# or
npm install
npm run dev
```

Open the frontend at `http://localhost:3000` (default for Next.js) and backend at `http://localhost:8080`.

API endpoints (quick list)

- `GET /api/users` — list users
- `GET /api/users/{id}` — get user
- `POST /api/users` — create user
- `PUT /api/users/{id}` — update user
- `DELETE /api/users/{id}` — delete user

Same patterns exist for `/api/orders`, `/api/menu-items`, `/api/ingredients`, `/api/feedbacks`, `/api/complaint-logs`, etc. See controllers in `backend/src/main/java/com/restroaudit/backend/controller`.

VIVA-ready Q&A (short paste-ready answers)

- What is ORM? — Object Relational Mapping maps Java classes (`@Entity`) to DB tables; JPA/Hibernate generates SQL automatically.
- How are endpoints defined? — In controllers via `@RestController` with `@GetMapping/@PostMapping` etc.; controllers call services which call repositories.
- Where is DB configured? — `backend/src/main/resources/application.properties` (contains `spring.datasource.url`, user, password).
- How does frontend send branch-scoped requests? — `lib/api.ts` reads `restroaudit.currentUser` from `localStorage` and sends `X-Branch-Id` header.

Troubleshooting / Tips

- If backend can't connect to DB: check credentials in `application.properties`, ensure MySQL is running and database exists.
- To see SQL statements: `spring.jpa.show-sql=true` is already enabled.
- If you add a new DB table and want JPA to manage it: create an `@Entity` and repository; restart the app.

Where to edit if you want to add a feature

- New endpoint → create controller method in appropriate controller or new controller under `backend/.../controller`.
- Business logic → add to a `@Service` or update existing service.
- Persisted model → add `@Entity` under `backend/.../model` and a `JpaRepository`.
- Frontend usage → add a helper in `frontend/lib/api.ts` and call it from a page under `frontend/app`.

If you'd like, I can: (pick one)

- also commit a short `README.md` at repo root summarizing these steps, or
- redact the DB password in `application.properties` and replace with an environment-variable example.

File created: `docs/PROJECT_GUIDE.md`